白盒加密SDK

一、产品简介

二、iOS版本

2.1 核心功能

2.2 环境需求

2.3 项目接入

2.4 使用说明

三、Android版本

3.1 核心功能

3.2 环境需求

3.3 SDK接入

3.4 Android Studio 集成

3.5 React Native 方式集成

3.6 DXWhiteBox类说明

3.7 使用说明

3.8 问题错误汇总

四、Server Java

4.1 功能介绍

4.2 环境需求

白盒加密SDK

一、产品简介

白盒加密SDK是顶象技术开发的一款面向移动应用客户端安全保护的产品。旨在帮助客户端低成本接入高标准的安全保护机制，免受恶意安全攻击，从而集中精力建设业务本身。

二、iOS版本

2.1 核心功能

- 密钥保护
  - 在安全虚机中保护密钥
- 安全加解密
  - 在安全虚机中实现加解密过程，保证加解密安全性
- 安全文件存储
  - 在安全虚机中实现文件的加密保存
- 网络通讯数据安全
  - 通讯链路保护
    - 对客户端发出的数据加签，保证服务器端接收到的数据真实性
- 运行时安全
  - 防重打包
    - 绑定包名，防止盗版、广告插入、恶意代码注入等重打包行为
  - 调试器监测
    - 防止程序被攻击者调试
  - 设备越狱监测

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

- 检测当前进程是否被代码注入
- 代码hook监测
  - 检测当前进程API是否被hook

2.2 环境需求

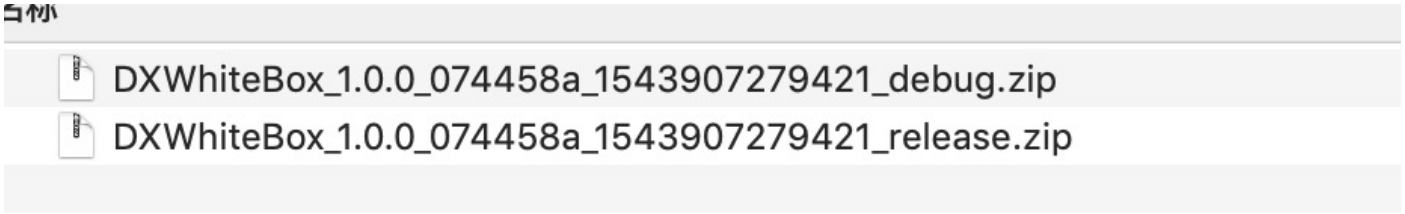
条目	说明
兼容平台	iOS 8.0+
开发环境	XCode 4.0 +
CPU架构	armv7, arm64, i386, x86_64
SDK依赖	libz, libresolv, libc++

SDK Framework体积正常约在10M左右，最低限度能压缩在4M左右（非bitcode版本），如有需要请咨询我们客服热线

2.3 项目接入

2.3.1 SDK获取

- 从顶象技术官网创建白盒加密SDK应用，然后下载SDK。下面是SDK的目录结构：



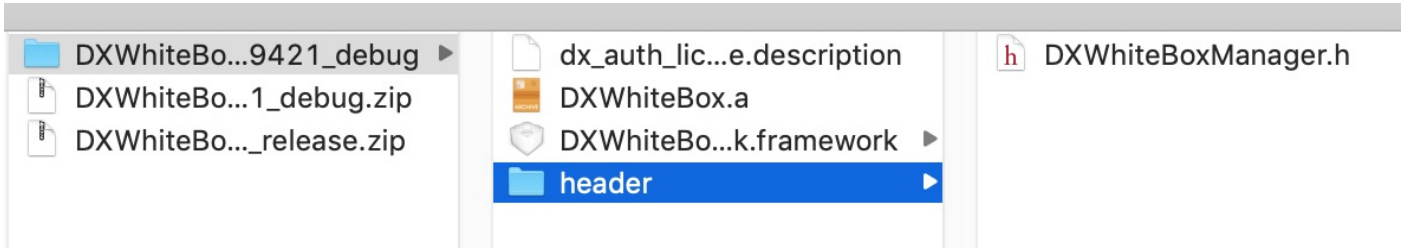
`DXWhiteBox_xxx_xxx_xxx_debug.zip` 白盒debug 授权集成库 `DXWhiteBox_xxx_xxx_xxx_release.zip` 白盒release 授权集成库

- 解压 `DXWhiteBox_xxx_xxx_xxx_xxx.zip` 文件，得到以下文件

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

- `Header/DXWhiteBoxManager.h` 头文件
- `dx_auth_license.description` 授权描述文件
- `DXWhiteBoxFramework.framework` 已授权framework 集成库
- `dx_white_box_server_license.json` license server 描述文件，需配合KeyTools.java使用

例如图：



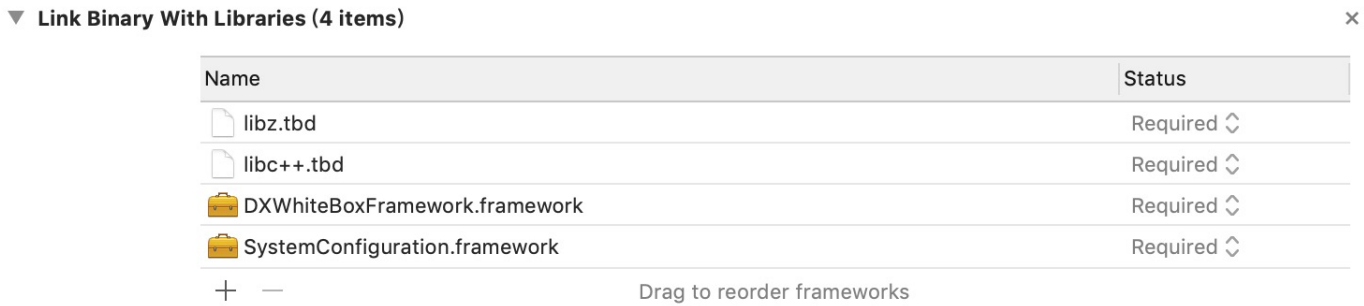
2.3.2 将SDK接入XCode

导入静态库及头文件

将SDK目录（包含静态库及其头文件）直接拖入工程目录中，或者右击总文件夹添加文件。 或者 将 DXWhiteBoxFramework.framework 拖进framework存放目录。 以下选取其一即可

集成FrameWork到工程

选择 `Build Phases -> Link Binary With Libraries` ， 点击加号， 添加 `libz.tbd` , `libc++.tbd` , `SystemConfiguration.framework` 以及 `DXWhiteBoxFramework.framework` 如下图所示：



集成静态库.a文件

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

SystemConfiguration.framework 以及 DXWhiteBox.a ，如下图所示：

▼ **Link Binary With Libraries (4 items)**

Name	Status
libz.tbd	Required
libc++.tbd	Required
SystemConfiguration.framework	Required
DXWhiteBox.a	Required

+ − Drag to reorder frameworks

2.4 使用说明

2.4.1 安全加解密使用说明

k0~k9使用说明

默认情况下，stee_license.dx会随机生成11种加密加签密钥，并在应用首次使用启动后随机生成一个AES密钥。详细介绍如下所示：

名字	使用说明	算法	功能
k0	每台设备随机生成，用于本地资源加解密	AES	加密，解密
k1	白盒AES算法，性能优于普通AES算法，适用于各种情况	WBAES	加密，解密，加签，验签
k2	SHA1哈希算法，适用于网络数据加签验签	SHA1	加签，验签
k3	SHA256哈希算法，适用于网络数据加签验签	SHA256	加签，验签
k4	普通的AES算法，适用于各种情况	AES	加密，解密，加签，验签
k5	RC4算法，适用于各种情况	RC4	加密，解密，加签，验签

名字 使用说明 算法 功能

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

k7	MD5哈希算法，适用于网络数据加签验签	MD5	加签，验签
k8	国密SM3哈希算法，适用于网络数据加签验签	SM3	加签，验签
K9	国密SM4对称加密算法，适用于各种情况	SM4	加密，解密，加签，验签
K-Random	顶象随机算法，适用于各种情况	Random	加密，解密，加签，验签

方法和参数说明

```
// 安全组件：DXWhiteBoxManager
@interface DXWhiteBoxManager : NSObject

/**
 开启证书指定项的保护,如反调试，反hook

 */
+ (void)protection;

/**
 数据加密

 @param input 待加密数据
 @param keyAlias key别名，如k0, k1, k2等等
 @return 加密后数据
 */
+ (NSData *)encrypt:(NSData *)input withKey:(NSString *)keyAlias;

/**
 数据解密

 @param input 待解密数据
 @param keyAlias key别名，如k0, k1, k2等等
 @return 解密后数据
 */
+ (NSData *)decrypt:(NSData *)input withKey:(NSString *)keyAlias;
```

4.4 密钥描述文件
白盒加密SDK
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
 @param data 待加签的数据
 @param keyAlias key别名，如k1, k2等等
 @return 数据签名
 */
+ (NSString *)sign:(NSData *)data withKey:(NSString *)keyAlias;

/**
 数据验签

 @param data 待验签的数据
 @param keyAlias key别名，如k1, k2等等
 @param sig 数据签名
 @return 数据签名是否验证通过
 */
+ (BOOL)verify:(NSData *)data withKey:(NSString *)keyAlias andSig:(NSString *)sig;


/****以下New 加解密加签验签部分使用K-Random 进行加解密******/
/**
 *(NEW) 数据加密
 @param input 待加密数据
 @return 加密后数据
 */
+ (NSData *)encrypt:(NSData *)input;

/**
 *(NEW)数据解密

 @param input 待解密数据
 @return 解密后数据
 */
+ (NSData *)decrypt:(NSData *)input;

/**
 *(NEW)数据加签

 @param data 待加签的数据
 @return 数据签名
 */
```

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
 *(NEW)数据验签

 @param data 待验签的数据
 @param sig 数据签名
 @return 数据签名是否验证通过
 */
+ (BOOL)verify:(NSData *)data  andSig:(NSString *)sig;

@end
```

2.4.3 使用示例

```
//开启相关保护
[DXWhiteBoxManager protection];

// 待加密/加签的数据
NSData *testData = [@"HelloWorld" dataUsingEncoding:NSUTF8StringEncoding];
// 测试k0， 只能用于加解密
NSString *key = @"k0";
NSData *testDataEncrypted = [DXWhiteBoxManager encrypt:testData withKey:key];
NSData *testDataDecrypted = [DXWhiteBoxManager decrypt:testDataEncrypted withKey:key];
NSAssert([testData isEqualToData:testDataDecrypted], @"key[%@] encrypt/decrypt fails: '%@' != '%@'.", key

// 测试k1， k1为单向密钥，由k1加密/加签的数据只能由服务端解密/验签，反之亦然
key = @"k1";
testDataEncrypted = [DXWhiteBoxManager encrypt:testData withKey:key];
NSAssert(testDataEncrypted != nil, @"key[%@] encrypt fails.", key);
NSAssert([DXWhiteBoxManager sign:testData withKey:key] != nil, @"key[%@] sign fails.", key);

// 测试k2-k9
NSArray<NSString *> *keys = @[@"k2", @"k3", @"k4", @"k5", @"k6", @"k7", @"k8", @"k9"];
for(NSString* key : keys) {
    NSString *sig = [DXWhiteBoxManager sign:testData withKey:key];
    NSAssert([DXWhiteBoxManager verify:testData withKey:key andSig:sig], @"key[%@] sign/verify fails.", k
}

// NEW 测试默认使用K-Random加解密方法
 NSData *testDataNewEncrypted = [DXWhiteBoxManager encrypt:testData];
```

4.4 密钥描述文件
白盒加密SDK
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

2.4.4“安全文件存储”使用说明

方法和参数说明

```
/**
 将Data加密到文件,完成后通过encryptSuccess:WithDestinationURL:error:回调

 @param sourceData 数据
 @param destination 加密文件URL
 @param keyAlias key别名，如k1, k2等等
 @param delegate delegate
 */
+ (void)encryptWithSourceData:(NSData *)sourceData toDestinationURL:(NSURL *)destination withKey:(NSStrin

/**
 将文件数据解密到Data,完成后通过decryptSuccess:outPutData:WithSourceURL:error:回调

 @param source 解密文件URL
 @param keyAlias key别名，如k1, k2等等
 @param delegate delegate
 */
+ (void)decryptWithSourceURL:(NSURL *)source withKey:(NSString *)keyAlias andDelegate:(id<DXWhiteBoxManag

@end
```

文件安全存储使用示例

```
//数据->文件加解密(仅限"k0","k4","k5","k6","k9")
 [DXWhiteBoxManager encryptWithSourceData:data toDestinationURL:destinationURL withKey:key andDelegate:de

 [DXWhiteBoxManager dncryptWithSourceURL:sourceURL withKey:key andDelegate:delegate];

//加密回调
- (void)encryptSuccess:(BOOL)isSuccess WithDestinationURL:(NSURL *)destinationURL error:(NSError *)error{
 //do something
```

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
- (void)decryptSuccess:(BOOL)isSuccess outPutData:(NSData *)data withSourceURL:(NSURL *)sourceURL error:(
 //do something
}
```

2.4.5 FAQ

1: debug 和 release 包的区别

- debug： 在测试环境测试使用， 不会检测debug，注入，hook 等恶意环境
- release： 在生产环境使用， 会检测debug，注入，hook 等恶意环境，检测到了会闪退

三、Android版本

3.1 核心功能

- 密钥保护
  - 在安全虚机中保护密钥
- 安全加解密
  - 在安全虚机中实现加解密过程，保证加解密安全性
- 安全文件存储
  - 在安全虚机中实现文件的加密保存
- 网络通讯数据安全
  - 通讯链路保护
    - 对客户端发出的数据加签，保证服务器端接收到的数据真实性
- 运行时安全
  - 防重打包
    - 绑定签名和包名，防止盗版、广告插入、恶意代码注入等重打包行为
  - 内存保护
    - 防止内存被拷贝
  - 调试器监测
    - 防止程序被攻击者调试

白盒加密SDK

一、产品简介
二、iOS版本
2.1 核心功能
2.2 环境需求
2.3 项目接入
2.4 使用说明
2.5 Java接口说明
2.6 KeyTools 使用
2.7、非Java开发环境
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求
4.3 密钥描述文件
4.4 密钥描述文件
4.5 Java接口说明
4.5.1 加签接口
4.5.2 加解密接口
4.6 KeyTools 使用
4.7、非Java开发环境
五、FAQ

- 代码注入监测
  - 检测当前进程是否被代码注入
- 代码hook监测
  - 检测当前进程API是否被hook
- Android模拟器检测)
  - 检测所有主流Android模拟器
- Android app多开监测
  - 检测app是否处于多开运行环境

3.2 环境需求

条目	说明
兼容平台	Android 4.0+
开发环境	Android Studio 3.0.1 或者 Eclipse + ADT
CPU架构	ARM 或者 x86
SDK三方依赖	无

3.3 SDK接入

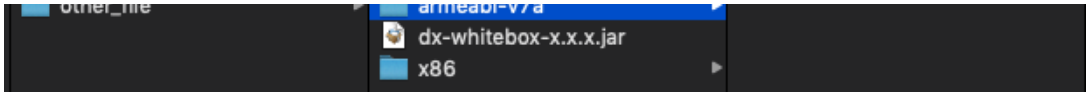
3.3.1 SDK获取

1. 访问顶象技术官网，注册账号
2. 登录, 进入控制台，访问“全流程端防控->白盒加密SDK”模块
3. 新增App，填写相关信息
4. 下载对应平台SDK

> PS：如找不到请联系技术人员

3.3.2 SDK文件结构

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

- SDK目录结构 

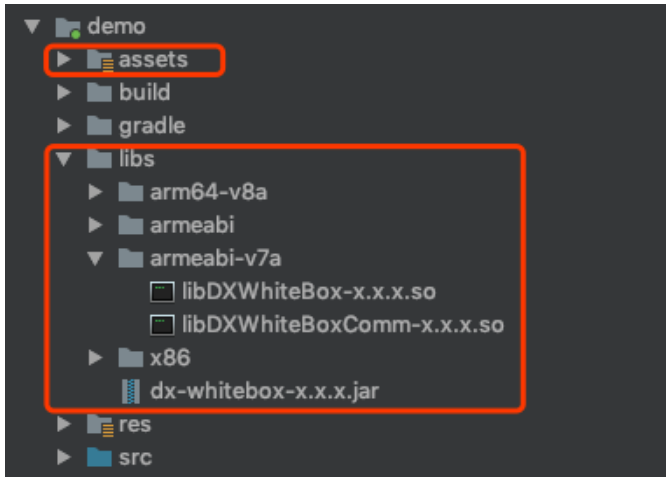
  - `assets` Android 资源文件
  - `dx-whitebox-${version}.jar` Android jar包
  - `armeabi` , `armeabi-v7a` , `arm64-v8a` , `x86` 4个abi平台的动态库文件

3.4 Android Studio 集成

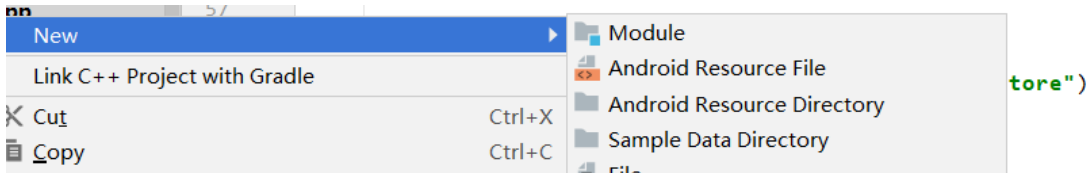
点击下载Demo

3.4.1 Android Studio导入jar, so

- 将dx-whitebox-x.x.x.jar, so文件放到相应模块的libs目录下



- 将assets中资源文件放到相应项目assets目录下
- 没有assets目录的，在Android Studio对应app目录下新建Assets Folder



白盒加密SDK

一、产品简介
二、iOS版本
2.1 核心功能
2.2 环境需求
2.3 项目接入
五、FAQ
2.4 使用说明

三、Android版本

3.1 核心功能

3.2 环境需求

3.3 SDK接入

3.4 Android Studio 集成

3.5 React Native 方式集成

3.6 DXWhiteBox类说明

3.7 使用说明

3.8 问题错误汇总

四、Server Java

4.1 功能介绍

4.2 环境需求

4.4 密钥描述文件

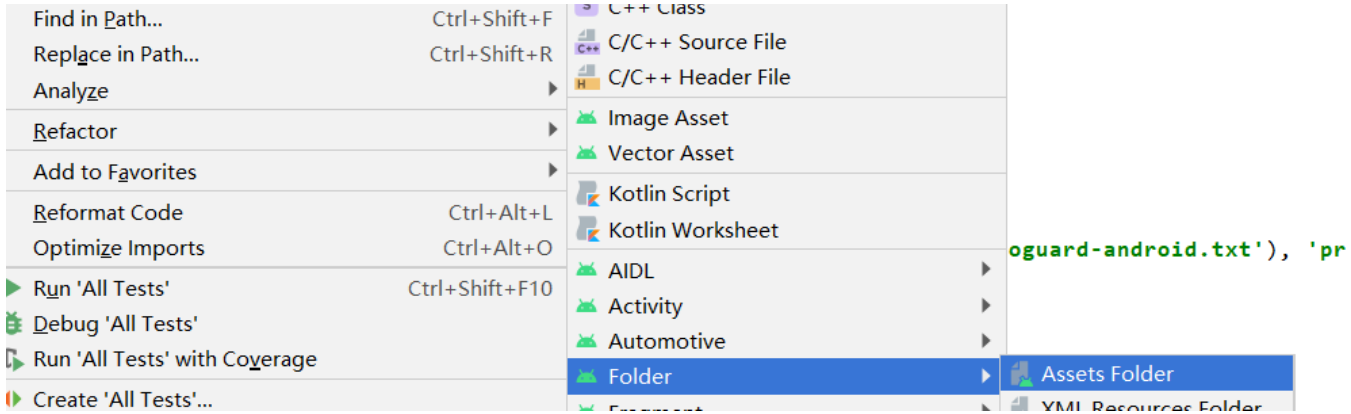
4.5 Java接口说明

4.5.1 加签接口

4.5.2 加解密接口

4.6 KeyTools 使用

4.7、非Java开发环境



3.4.2 build.gradle 配置

```
android{
    sourceSets {
        main {
            jniLibs.srcDirs = ['libs']
        }
    }

    packagingOptions {
        doNotStrip "**/libDX*.so"
    }

}
repositories{
        flatDir{
              dirs 'libs'
      }
}

dependencies {
    implementation fileTree(dir: 'libs', include: ['*.jar'])
}
```

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
-dontwarn *.com.dingxiang.mobile.**
-dontwarn *.com.mobile.strenc.**
-keep class com.dingxiang.mobile.whitebox.**{*;}
-keep class com.security.inner.**{*;}
-keep class *.com.dingxiang.mobile.**{*;}
-keep class *.com.mobile.strenc.**{*;}
```

3.5 React Native 方式集成

点击下载Demo

3.5.1 React Native 环境配置

具体参考 `facebook` 官方文档： https://facebook.github.io/react-native/docs/getting-started

3.5.2 集成SDK

- 通过官方 `react-native init {project_name}` 命令，可生成 `Android Studio` 项目，其目录在： `ReactNative` 根目录的 `android` 目录下
- `Android Studio` 项目如何集成SDK，请参考 `3.3 Android Studio 集成`

3.5.3 引入React Native库

build.gradle下：

```
dependencies {
    implementation fileTree(dir: "libs", include: ["*.jar"])
    implementation "com.android.support:appcompat-v7:${rootProject.ext.supportLibVersion}"
    implementation "com.facebook.react:react-native:+"  // From node_modules
}
```

3.5.4 添加React Native Android代码

- 创建 ReactContextBaseJavaModule 实现类

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
public class DXModule extends ReactContextBaseJavaModule {

    public DXModule(ReactApplicationContext reactContext) {
        mContext = reactContext;
    }

    @Override
    public String getName() {
        return "DXModule";
    }

    @ReactMethod
    public void startDemoTest(int param1, final Callback callback){
        // 调用SDK的地方，具体例子请查看Demo
    }

    ...
}
```

- 创建 ReactPackage 实现类

其中 `createNativeModules` 方法要返回 `DXModule` 的实例

```
public class DXPackage implements ReactPackage {

    @Override
    public List<NativeModule> createNativeModules(ReactApplicationContext reactContext) {
        List<NativeModule> modules = new ArrayList<>();
        modules.add(new DXModule(reactContext));
        return modules;
    }

    @Override
    public List<ViewManager> createViewManagers(ReactApplicationContext reactContext) {
        return Collections.emptyList();
    }
}
```

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

其中 `getPackages` 方法要返回 `DXPackage` 的实例

```
public class MainApplication extends Application implements ReactApplication {

  private final ReactNativeHost mReactNativeHost = new ReactNativeHost(this) {
    @Override
    public boolean getUseDeveloperSupport() {
      return BuildConfig.DEBUG;
    }

    @Override
    protected List<ReactPackage> getPackages() {
      return Arrays.<ReactPackage>asList(
              new MainReactPackage()
              , new DXPackage()
      );
    }

    @Override
    protected String getJSMainModuleName() {
      return "index";
    }
  };

  @Override
  public ReactNativeHost getReactNativeHost() {
    return mReactNativeHost;
  }

  @Override
  public void onCreate() {
      super.onCreate();
      SoLoader.init(this, /* native exopackage */ false);
  }
}
```

- 创建 ReactActivity 实现类

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
public class MainActivity extends ReactActivity {

    /**
     * Returns the name of the main component registered from JavaScript.
     * This is used to schedule rendering of the component.
     */
    @Override
    protected String getMainComponentName() {
        return "DXWhiteboxRNDemo";
    }
}
```

3.5.5 AndroidManifest 配置

```
<uses-permission android:name="android.permission.INTERNET" />
    <uses-permission android:name="android.permission.SYSTEM_ALERT_WINDOW"/>

    <application
      android:name=".MainApplication"
      android:label="@string/app_name"
      android:icon="@mipmap/ic_launcher"
      android:allowBackup="false"
      android:theme="@style/AppTheme">
      <activity
        android:name=".MainActivity"
        android:label="@string/app_name"
        android:configChanges="keyboard|keyboardHidden|orientation|screenSize"
        android:windowSoftInputMode="adjustResize">
        <intent-filter>
            <action android:name="android.intent.action.MAIN" />
            <category android:name="android.intent.category.LAUNCHER" />
        </intent-filter>
      </activity>
      <activity android:name="com.facebook.react.devsupport.DevSettingsActivity" />
    </application>
```

3.5.6 React Native调用Android代码

白盒加密SDK
一、产品简介
二、iOS版本
2.1 核心功能
2.2 环境需求
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求
4.4 密钥描述文件
4.5 Java接口说明
4.5.1 加签接口
4.5.2 加解密接口
4.6 KeyTools 使用
4.7、非Java开发环境

```
{
  "name": "DXWhiteboxRNDemo",
  "displayName": "DXWhiteboxRNDemo"
}
```

- 修改JS主界面 修改文件 `{RactNative_pro}/App.js` :

```
import React, {Component} from 'react';
import {Button, DeviceEventEmitter, NativeModules, View} from 'react-native';

export default class NativeInteractionDemo extends Component {
    // 注册监听
    componentDidMount() {
        this.subscription = DeviceEventEmitter.addListener('DXEvent', function (msg) {
            alert(msg)
        });
    }

    componentWillUnmount() {
        this.subscription.remove();
    }

    render() {
        return <View>
            <Button
                onPress={() => {
                    this.startDemo();
                    this.showToast();
                }
                }
                title="开始Demo"
            />
        </View>
    }

    // 调用安卓原生代码并回传数据
    startDemo() {
        NativeModules.DXModule.startDemoTest(1, (status, msg) => {
```

4.4 密钥描述文件
白盒加密SDK
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
    }

    // 调用原生安卓土司
    showToast() {
        NativeModules.DXModule.showToast("正在执行测试...")
    }
}
```

3.5.7 运行Android项目

ReactNative目录下，执行命令

```
react-native run-android
```

3.6 DXWhiteBox类说明

在调用功能方法之前必须调用初始化方法

3.6.1 SDK初始化

```
/**
 * @description SDK全局初始化，一般放在应用启动处调用，如Application.onCreate里。
 * 调用setup()方法后，会在当前app内启用：内存保护、调试器监测、设备root监测、
 * 代码注入监测、代码hook监测、Android模拟器检测、Android app多开监测等运行时安全检测。
 *
 * @param context Android上下文
 * @return 初始化是否成功
 */
public static boolean DXWhiteBox.setup(Context context);
```

3.6.2 数据加密/解密

```
/**
 * @description 加密
```

白盒加密SDK

一、产品简介
二、iOS版本
2.1 核心功能
2.2 环境需求
2.3 项目接入
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求
4.4 密钥描述文件
4.5 Java接口说明
4.5.1 加签接口
4.5.2 加解密接口
4.6 KeyTools 使用
4.7、非Java开发环境
五、FAQ

```
 */
public static byte[] encrypt(String key, byte[] input);

/**
 * @description 解密
 * @param key 指定解密方式：k0~k9。具体说明请看“k0~k9使用说明”
 * @param input 待解密数据
 * @return 解密后的数据
 */
public static byte[] decrypt(String key, byte[] input);

/**
 * @description 无key加密，内部随机
 * @param input 待加密数据
 * @return 加密后的数据
 */
public static byte[] encrypt( byte[] input);

/**
 * @description 无key解密，内部随机
 * @param input 待解密数据
 * @return 解密后的数据
 */
public static byte[] decrypt( byte[] input);
```

3.6.3 文件加密/解密

```
/**
 * @description 解密方式读取文件
 * @param key 指定解密方式：目前允许的key为k0, k4, k5, k6, k9。具体说明请看“k0~k9说明”
 * @param filePath 文件路径
 * @return 文件内容
 * @throws IOException
 */
public static byte[] readFile(String key, String filePath) throws IOException
```

白盒加密SDK

4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
 * @param data 写入数据
 * @param filePath 目标文件路径
 * @throws IOException
 */
public static void writeFile(String key, byte[] data, String filePath) throws IOException
```

3.6.4 加签/验证

```
/**
 * @description 加签数据
 * @param key 加签方式，k0 ~ k9。具体说明请看“k0~k9使用说明”
 * @param input 加签数据
 * @return 加签输出数据
 */
public static String sign(String key, byte[] input);


/**
 * @description 验签
 * @param key 验签方式，k0 ~ k9。具体说明请看“k0~k9使用说明”
 * @param input 验签数据
 * @param sig 加签值
 * @return 验签是否成功
 */
public static boolean verify(String key, byte[] input, String sig);

/**
 * @description 随机加签，不需要指定加签方式
 * @param input 待加签数据
 * @return 加签数据
 */
public static String sign( byte[] input);

/**
 * @description 随机验签，不需要指定加签方式
 * @param input 待验签数据
 * @param sig 加签值
 * @return 验签是否成功
 */
```

白盒加密SDK
一、产品简介
二、iOS版本
2.1 核心功能
2.2 环境需求
2.3 项目接入
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求
4.4 密钥描述文件
4.5 Java接口说明
4.5.1 加签接口
4.5.2 加解密接口
4.6 KeyTools 使用
4.7、非Java开发环境
五、FAQ

3.7 使用说明

3.7.1 “安全加解密”使用说明

3.7.1.1 k0~k9使用说明

默认情况下，SDK会随机生成11种加密加签密钥，并在应用首次使用启动后随机生成一个AES密钥。详细介绍如下所示：

名字	使用说明	算法	功能
k0	每台设备随机生成，用于本地资源加解密	AES	加密，解密
k1	白盒AES算法，性能优于普通AES算法，适用于各种情况	WBAES	加密，解密，加签，验签
k2	SHA1哈希算法，适用于网络数据加签验签	SHA1	加签，验签
k3	SHA256哈希算法，适用于网络数据加签验签	SHA256	加签，验签
k4	普通的AES算法，适用于各种情况	AES	加密，解密，加签，验签
k5	RC4算法，适用于各种情况	RC4	加密，解密，加签，验签
k6	XXTEA对称加密算法，适用于各种情况	XXTEA	加密，解密，加签，验签
k7	MD5哈希算法，适用于网络数据加签验签	MD5	加签，验签
k8	国密SM3算法，适用于网络数据加签验签	SM3	加签，验签
k9	国密SM4算法，适用于各种情况	SM4	加密，解密，加签，验签

名字 使用说明 算法 功能

4.4 密钥描述文件
白盒加密SDK
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

3.7.1.2 使用示例

```
// k0只能进行加解密
String key0 = "k0";
String test_data = "HelloWorld";

byte test_data_k0[] = DXWhiteBox.encrypt(key0, test_data.getBytes());
String test_data_old = new String(DXWhiteBox.decrypt(key0, test_data_k0));
if (!test_data.equals(test_data_old)) {
    throw new RuntimeException(String.format("key[%s] encrypt/verify fails", key0));
}

String keys[] = {"k2", "k3", "k4", "k5", "k6", "k7", "k8", "k9"};

for (String key : keys) {
    Log.i(TAG, String.format("Testing Key %s sign begin", key));
    String request_data_with_sig = DXWhiteBox.sign(key, wbaesTestData);
    Log.i(TAG, String.format("Testing Key %s sign finish", key));

    Log.i(TAG, String.format("Testing Key %s verify begin", key));
    boolean verify = DXWhiteBox.verify(key, wbaesTestData, request_data_with_sig);
    Log.i(TAG, String.format("Testing Key %s verify finish", key));

    if (!verify) {
        throw new RuntimeException(String.format("key[%s] 验签失败 : %d", key, wbaesTestData.length()));
    } else {
        Log.i(TAG, "key[" + key + "] 验签成功: " + wbaesTestData.length());
    }

}

//wbaes(白盒加解密只能用于在客户端加签，验签，以及加密)
String wbaesTestData = "haabhahaabhaabaa";
byte[] encryptData = DXWhiteBox.encrypt("k1", wbaesTestData.getBytes());

//客户端白盒加签
Map<String, String> request_data_with_sign = DXWhiteBox.sign("k1", request_data);
```

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
    Log.i(TAG, "客户端验签成功");
} else {
    Log.i(TAG, "客户端验签失败");
}
```

3.7.2 “安全文件存储”使用示例

```
String filePath = sdcardPath + "/Android/test.txt";
String data = "this is a test data";

try {

    DXWhiteBox.writeFile("k0", data.getBytes(), filePath);

    byte[] oData = DXWhiteBox.readFile("k0", filePath);

    Log.e(TAG, "=======data is >>>" + new String(oData));

} catch (Exception e) {
    e.printStackTrace();
}
```

3.8 问题错误汇总

错误码	错误描述
-1000 , -1101	授权失败，请联系顶象客服人员
-1001	SDK未授权，请查看‘SDK接入’
-1102	试用版本已过期，请联系顶象客服人员
-1103	包名或签名相关不正确，请校对包名/签名信息是否跟生成库提供的包名/签名一致
其他	设备运行在危险环境中，如注入、调试等环境

白盒加密SDK
一、产品简介
二、iOS版本
2.1 核心功能
2.2 环境需求
2.3 项目接入
2.4 使用说明
五、FAQ
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求
4.4 密钥描述文件
4.5 Java接口说明
4.5.1 加签接口
4.5.2 加解密接口
4.6 KeyTools 使用
4.7、非Java开发环境

4.1 功能介绍

Server SDK主要为接入顶象移动端安全SDK的客户提供服务端的SDK。包括在服务端的加解密，加签验签等功能。

4.2 环境需求

条目	说明
兼容平台	全支持
CPU架构	全支持
SDK三方依赖	无
SDK支持最低JDK版本	1.6

4.3 SDK接入

目前仅提供jar的方式接入，将jar引入到项目目录中即可。

4.4 密钥描述文件

在android/iOS 安全SDK的license文件压缩包中，包含dx_license.description文件，记录着license中随机生成的密钥，在进行加解密以及数据加签验签时需要用到。同时，请妥善保管，切勿掉失或外泄。

以及生成dx_white_box_server_license.json 文件，该文件保存为明文license key 数据，所以请勿修改内容。该文件配合在 KeyTools.java 中使用，使用请查看KeyTools 的使用用例。

4.5 Java接口说明

4.5.1 加签接口

4.5.1 说明

4.4 密钥描述文件
白盒加密SDK
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

```
该方法为对input数据进行签名

Parameters:
String input:
该参数为需要加签的数据

Return:
返回input数据的签名
```

```
public boolean verify(String input, String sign)

Description:
对数据进行验证

Parameters:
String input:
该参数为需要验证签名的数据
String sign:
该参数为需要对比的签名
```

4.5.1 使用示例

```
String data = "HelloWorld DX";

//从描述文件中提取k4的密钥
Key4 key4 = new Key4("xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx");
String sig = key4.sign(data.getBytes("UTF-8"));   //服务器加签
if(!key4.verify(data, sig)){
  //验签不成功

}

//以下是KeyRandom 的使用,参数1 位置为AppID ,  参数2 为AppSercert ,  参数3 为RandomKey
 KeyRandom randomKey = new KeyRandom("aaaaaaaaaaaaaa", "bbbbbbbbbbbb", "cccccccccccc");
String sig = randomKey.sign(data.getBytes("UTF-8"));
```

白盒加密SDK

一、产品简介
二、iOS版本
2.1 核心功能
2.2 环境需求
2.3 项目接入
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求
4.4 密钥描述文件
4.5 Java接口说明
4.5.1 加签接口
4.5.2 加解密接口
4.6 KeyTools 使用
4.7、非Java开发环境
五、FAQ

```
}
```

4.5.2 加解密接口

4.5.2 说明

```
public byte[] encrypt(byte[] input)

Description:
对数据进行加密，目前只有一种加密算法

Parameters:
String input:
该参数为需要加密的数据

Return:
返回加密后的数据
```

```
public byte[] decrypt(byte[] input)

Description:
对数据进行解密，目前只有一种加密算法

Parameters:
String input:
该参数为需要解密的数据

Return:
返回解密后的数据
```

4.5.2 使用示例

白盒加密SDK

一、产品简介
二、iOS版本
2.1 核心功能
2.2 环境需求
2.3 项目接入
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求
4.4 密钥描述文件
4.5 Java接口说明
4.5.1 加签接口
4.5.2 加解密接口
4.6 KeyTools 使用
4.7、非Java开发环境
五、FAQ

```
Key4 key4 = new Key4("xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx");
byte[] encrypt_data = key4.encrypt(data.getBytes("UTF-8"));   //服务器加签

if(!Arrays.equals(data.getBytes("UTF-8"), key4.decrypt(encrypt_data))){
//解密不成功

}

//以下是KeyRandom 的使用,参数1 位置为AppID ,  参数2 为AppSercert ,  参数3 为RandomKey
 KeyRandom randomKey = new KeyRandom("aaaaaaaaaaaaaa", "bbbbbbbbbbbb", "cccccccccccc");
        byte[] encrypt_random = randomKey.encrypt(data.getBytes(Charset.forName("UTF-8")));

if(!Arrays.equals(data.getBytes("UTF-8"), randomKey.decrypt(encrypt_data))){
//解密不成功

}
```

4.6 KeyTools 使用

以下是KeyTools 对应的key名称以及作用

名字	使用说明	算法	功能
k1	白盒AES算法，性能优于普通AES算法，适用于各种情况	WBAES	加密，解密，加签，验签
k2	SHA1哈希算法，适用于网络数据加签验签	SHA1	加签，验签
k3	SHA256哈希算法，适用于网络数据加签验签	SHA256	加签，验签
k4	普通的AES算法，适用于各种情况	AES	加密，解密，加签，验签
k5	RC4算法，适用于各种情况	RC4	加密，解密，加签，验签
k6	XXTEA对称加密算法，适用于各种情况	XXTEA	加密，解密，加签，验签

白盒加密SDK
4.4 密钥描述文件
4.5 Java接口说明
一、产品简介
4.5.1 加签接口
二、iOS版本
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
五、FAQ
2.4 使用说明
三、Android版本
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
四、Server Java
4.1 功能介绍
4.2 环境需求

k8	国密SM3哈希算法，适用于网络数据加签验签	SM3	加签，验签
K9	国密SM4对称加密算法，适用于各种情况	SM4	加密，解密，加签，验签
k-random	顶象随机算法，适用于各种情况	Random	加密，解密，加签，验签

代码上的使用：

```
//初始化KeyTools 工具，指定license 文件的路径

KeyTools.getSingleton().init("license_file"); // /xx/xx/dx_white_box_server_license.json

//然后使用KeyTools 的getKey 进行相关的Key 的使用，如上单独使用的情况,使用 k-random 请直接填入k-random
BaseKey key4 = KeyTools.getSingleton().getKey("k4");

String data = "HelloWorld DX";
String sig = key4.sign(data.getBytes("UTF-8"));   //服务器加签
if(!key4.verify(data, sig)){
  //验签不成功

}

byte[] encrypt_data = key4.encrypt(data.getBytes("UTF-8"));   //服务器加签
if(!Arrays.equals(data.getBytes("UTF-8"), key4.decrypt(encrypt_data))){
//解密不成功

}
```

4.7、非Java开发环境

如果后台所使用的并非Java开发环境，请联系顶象技术人员。

五、FAQ

白盒加密SDK

一、产品简介
二、iOS版本
五、FAQ
三、Android版本
四、Server Java
4.4 密钥描述文件
4.5 Java接口说明
4.5.1 加签接口
4.5.2 加解密接口
2.1 核心功能
4.6 KeyTools 使用
2.2 环境需求
4.7、非Java开发环境
2.3 项目接入
2.4 使用说明
3.1 核心功能
3.2 环境需求
3.3 SDK接入
3.4 Android Studio 集成
3.5 React Native 方式集成
3.6 DXWhiteBox类说明
3.7 使用说明
3.8 问题错误汇总
4.1 功能介绍
4.2 环境需求

快速入口
立即注册
立即登录
文档中心
顶象学院

了解顶象
公司简介
团队介绍
加入我们
联系我们

用户中心
企业认证
安全设置
账号管理
渠道商合作

合作支持
技术支持：
support@dingxiang-inc.com
市场合作：
marketing@dingxiang-inc.com
渠道商合作：
gang.xu@dingxiang-inc.com

员客栈 MottoIN CocoaChina 京东万象 顶象工控安全 微投票 多投票

Copyright © 2020 北京顶象技术有限公司 版权所有 京ICP备17 号